

CS & IT ENGINEERING

Programming in C

Arrays and Pointers

Lec- 07



By- Pankaj Sharma sir



TOPICS TO BE
COVERED



Arrays and Pointers

`int* p ;`

→

(p+1) →

`char* q ;`

=

`q+1`

```
int a = 10;
```

```
char *p;
```

```
p = (char*) &a;
```

```
printf("/d", *p);
```

⇒ little End.

⇒ Big End

void pointer

void *ptr;

int a = 10;

char b = 'A';

ptr = &a;

int *ptr;

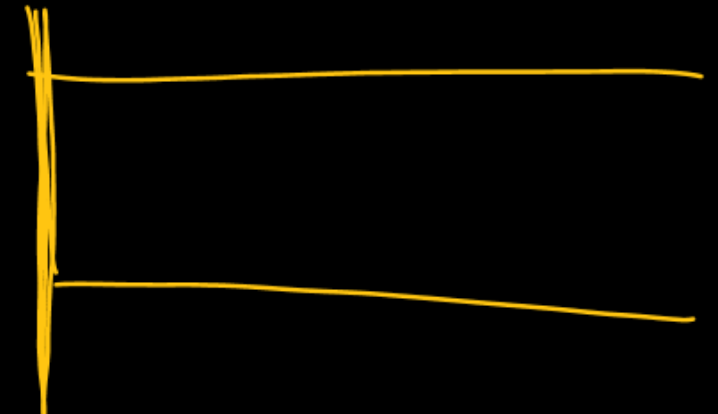
→
→
→

printf("%d", *ptr);

printf("%d", *ptr);

Compiler
will shout

ptr



Do not
dereference a void
pointer directly

```
void *Ptr;  
int a = 10;  
char b = 'A';  
Ptr = &a;
```

Problem $\Rightarrow$ *Ptr

```
printf("/d", *(int*)Ptr);  
                  ↓  
                  typecasting
```

```
void *ptr;  
int a = 10;  
char b = 'A';  
ptr = &a;  
  
printf("/d", *(int*)ptr);  
  
ptr = &b; ✓
```

```
printf("/d", *ptr);
```

→ Error

→ printf("/c", *(char*)ptr);

✓

↓ dereferencing

↓ typecasting

```
int a[4] = {10, 20, 30, 40};
```

```
void *ptr;
```

```
ptr = &a;
```

```
ptr = ptr + 1;
```



ptr + 1

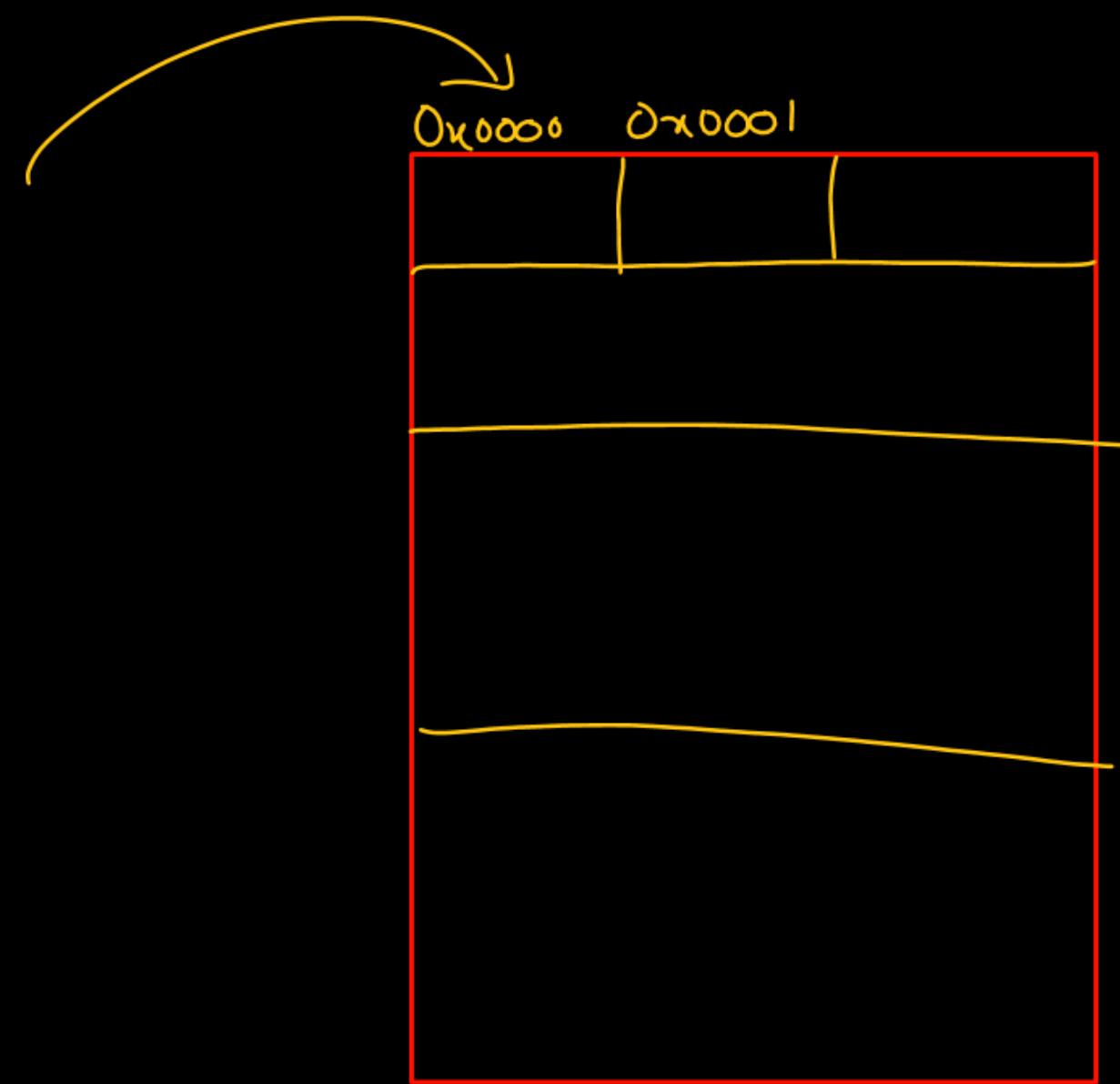
No arith. can
be performed
on void pointer

NULL Pointer

≥ 0

Pointer $\Rightarrow$ Address

Valid address



Dangling pointer

```
int *f(){
```

```
    static int x = 10;
```

```
    return &x;
```

```
}
```

```
void main(){
```

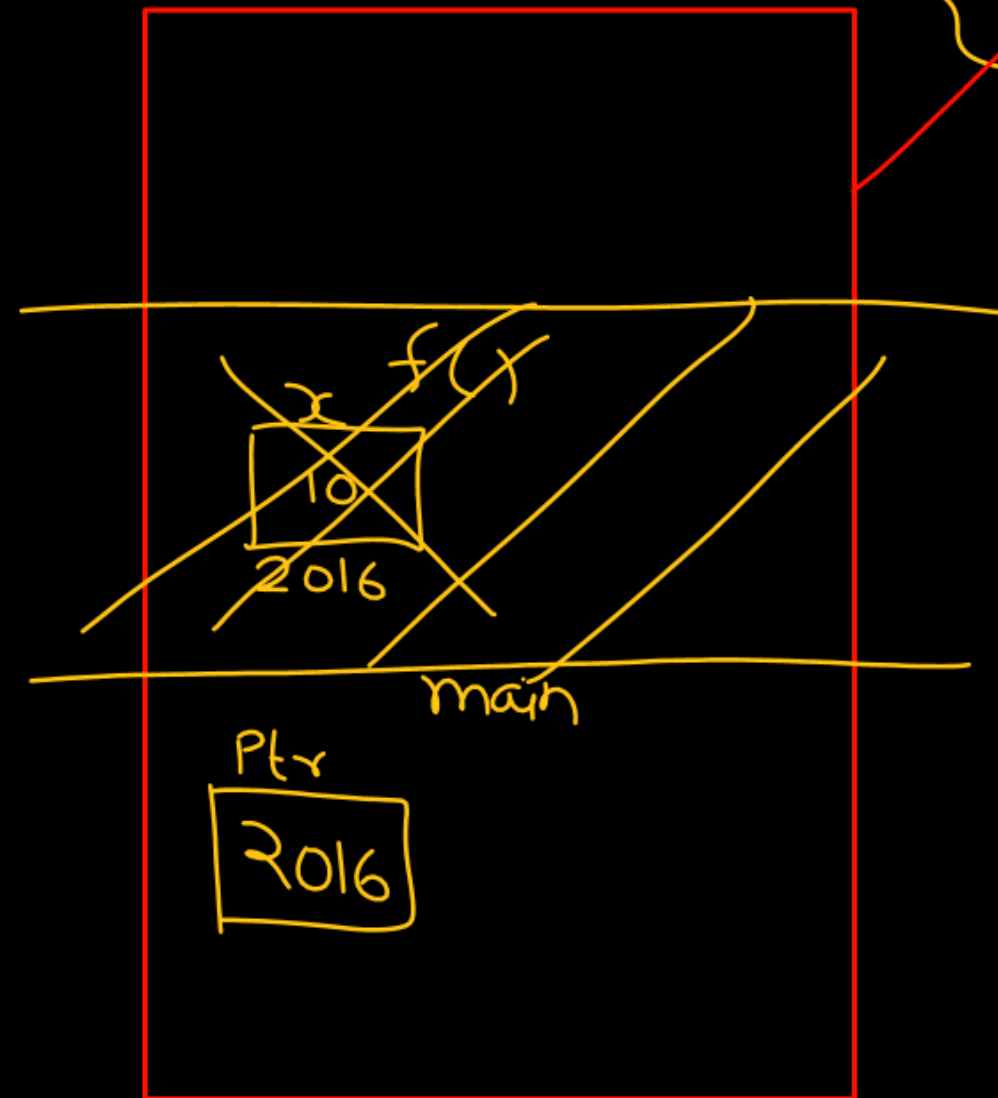
```
    int *ptr;
```

```
    ptr = f();
```

```
    printf("/d", *ptr);
```

```
}
```

ptr



2016

Wild Pointer

```
void main() {
```

```
    int a;
```

```
    printf("/d", a);
```

Garbage

a		
	Gar.	

```
    }
}
```

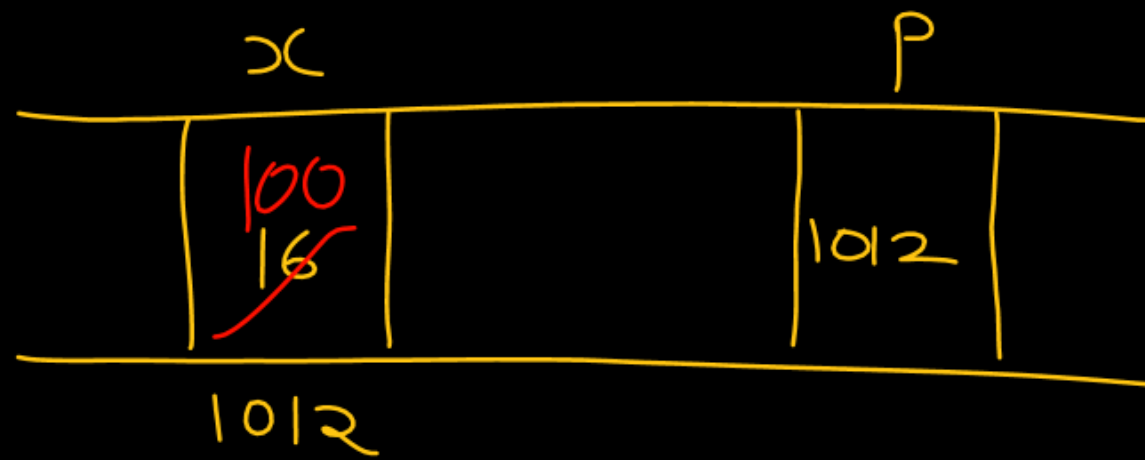
```
void main() {
```

```
    int *P;
```

uninitial.
pointer

pointer
variable

P		
	Garbage	



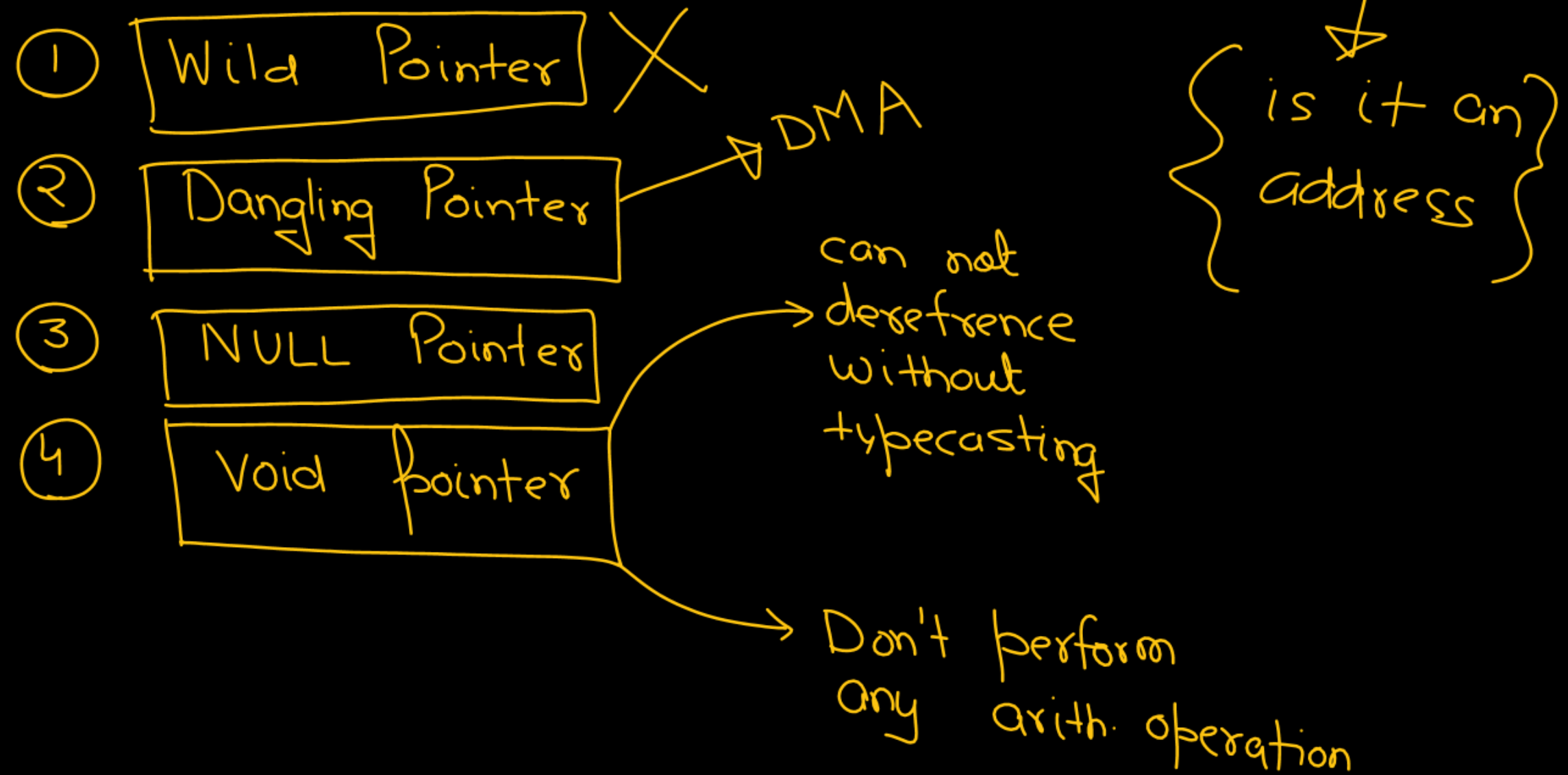
```
void main() {  
    int x = 16;  
    int *p;  
    ==  
    *p = 100;  
}
```

Garbage → Address

Wild Pointer : An initialized pointer.

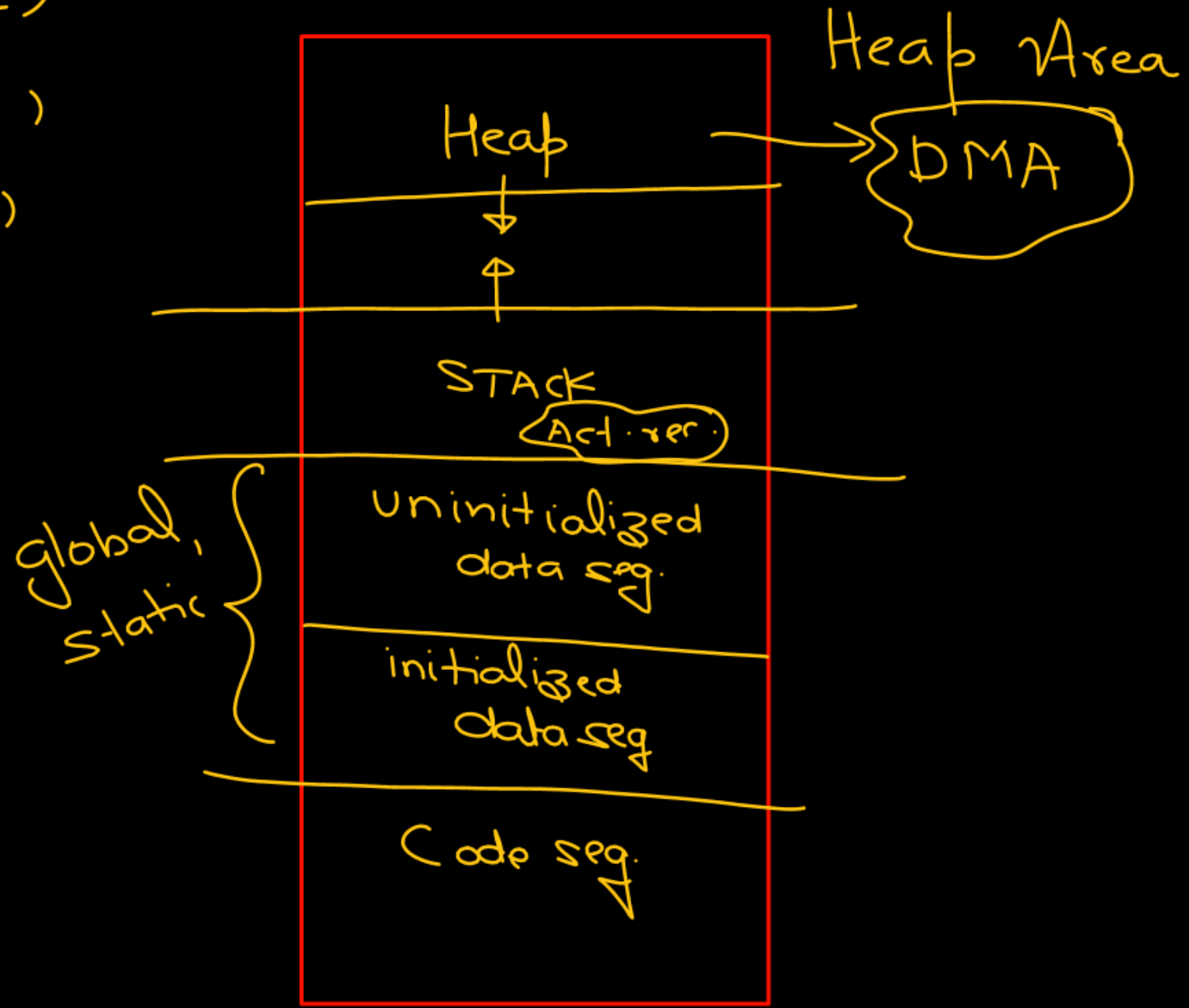
`int *p = (int*)0;`

`int *p = 0;`



Dynamic Memory Allocation

- (i) malloc()
- (ii) calloc()
- (iii) realloc()
- (iv) free



malloc

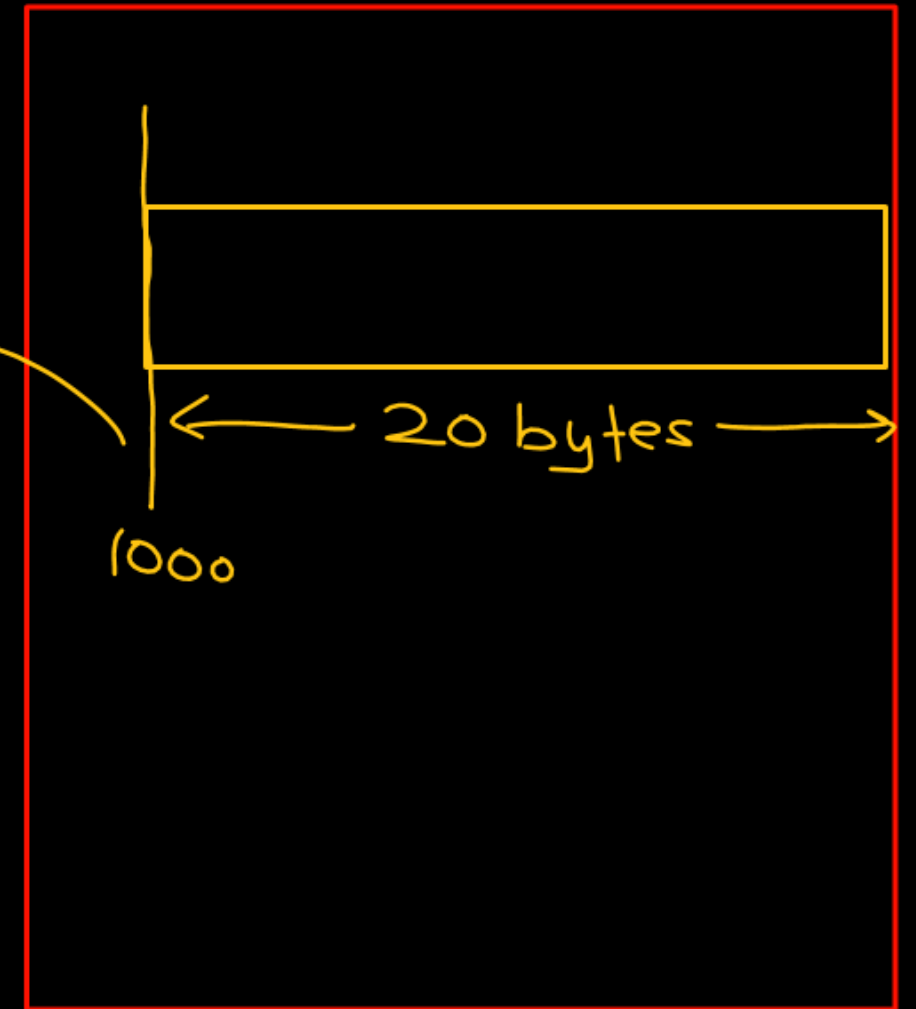
(i) malloc(size in bytes)

malloc(20)

malloc(60000)

malloc(-24)

(void*) malloc(unsigned int)



Syntax

$(void^*) malloc(\text{unsigned int})$

(ii) 5 int. $\rightarrow$ a
 $\searrow$ b

$\times$
 $\boxed{malloc(5 \times 4)}$

$\checkmark \checkmark$ $malloc(5 \times \text{sizeof(int)})$

`int *p;`

`p = malloc(5 * sizeof(int));`

`scanf("/d", p); 10`

`scanf("/d", p+1); 20`

`scanf("/d", p+2); 30`

`scanf("/d", p+3); 40`

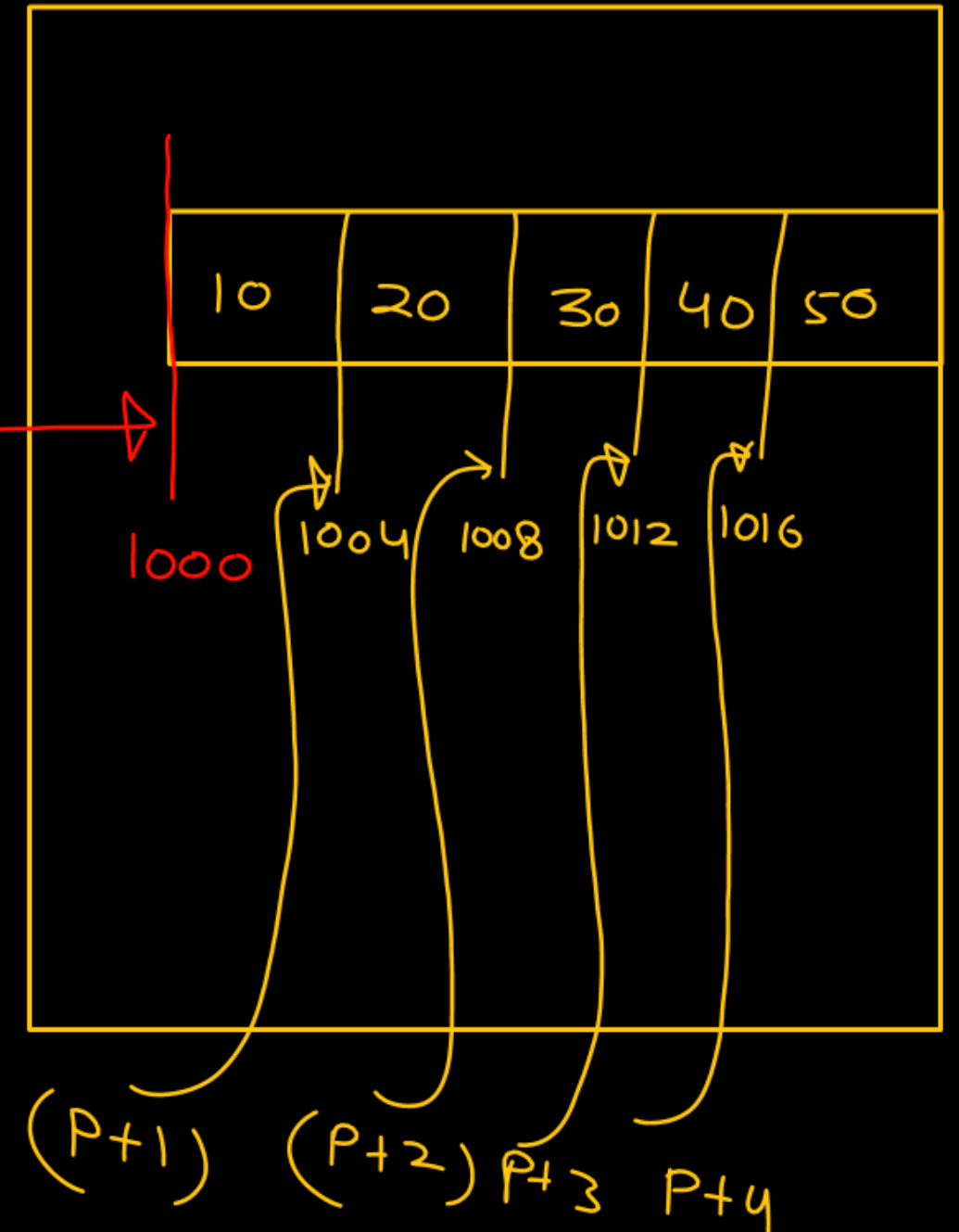
`scanf("/d", p+4); 50`

OR

`for(i=0; i<5; i++)`

`scanf("/d", p+i);`

1000
p



```
int *p;
```

```
p = malloc(5 * sizeof(int));
```

```
for(i=0; i<5; i++)
```

```
scanf("%d", p+i);
```

```
printf("%d", *(p+0)); 10
```

```
printf("%d", *(p+1)); 20
```

```
printf("%d", *(p+2)); 30
```

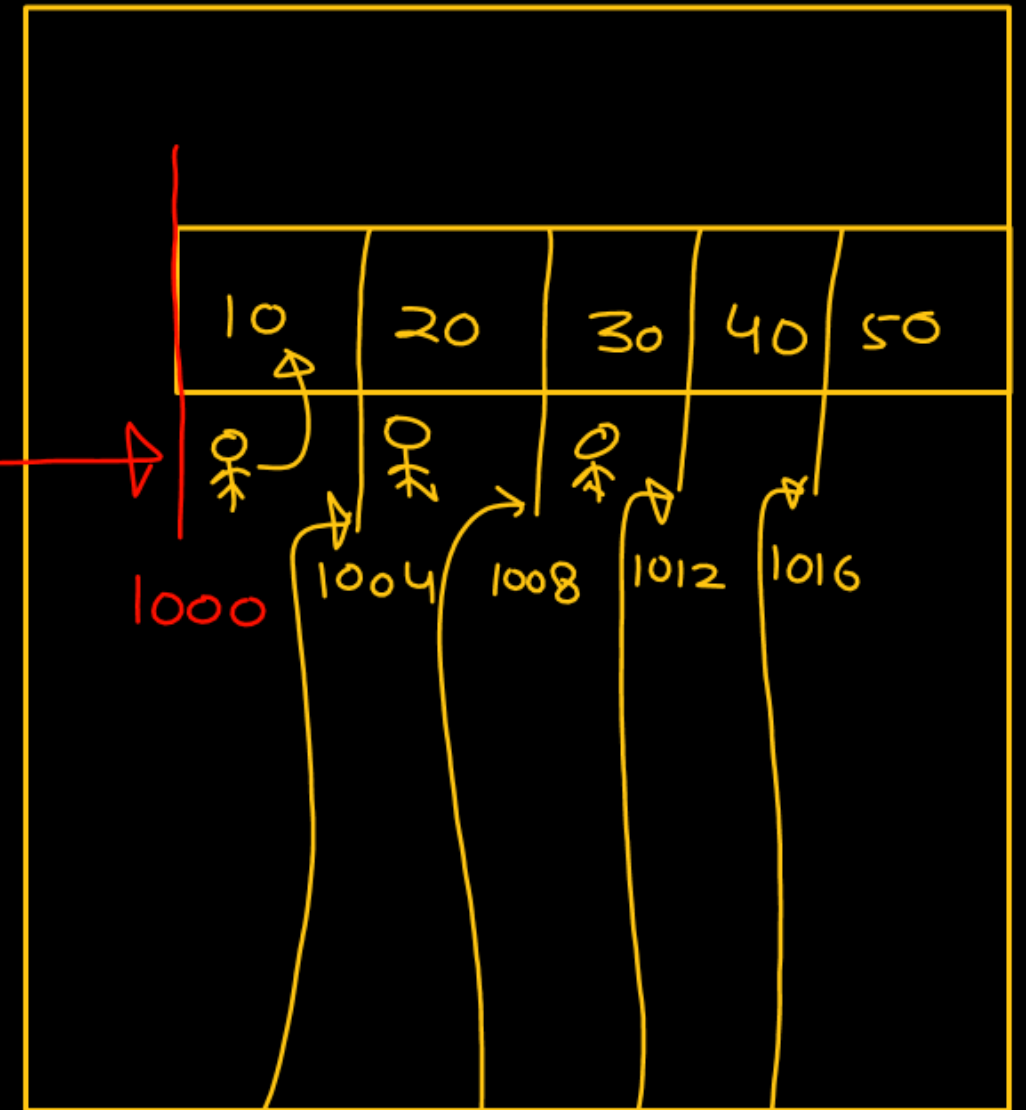
```
printf("%d", *(p+3)); 40
```

```
printf("%d", *(p+4)); 50
```

1000
p

```
for(i=0; i<5; i++)
```

```
printf("%d", *(p+i));
```



(p+1)

(p+2)

p+3

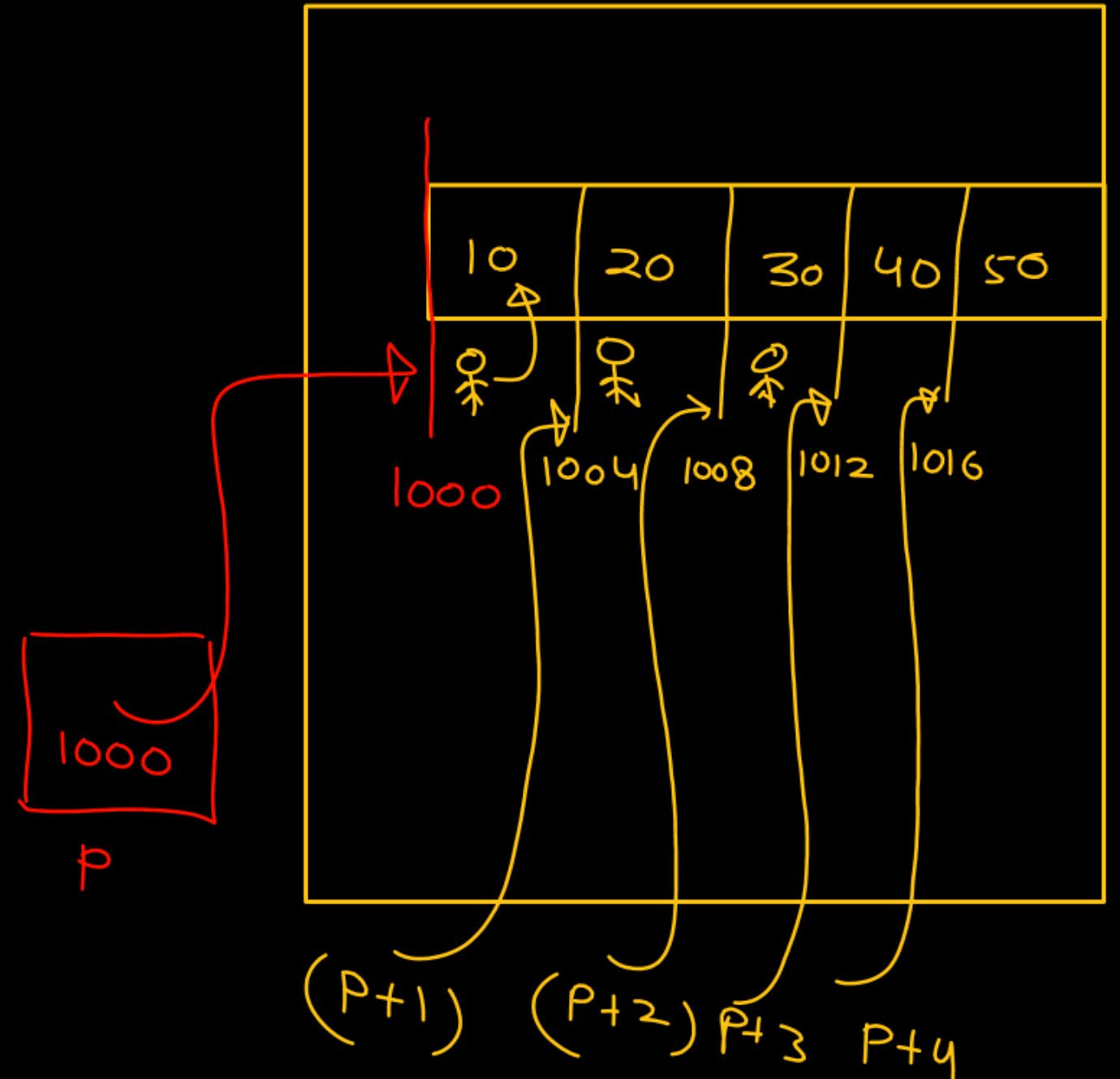
p+4

```
int *p;
```

```
p = malloc(105 * sizeof(int));
```

```
for(i=0; i<105; i++)  
    scanf("/d", p+i);
```

```
for(i=0; i<105; i++)  
    printf("/d", → P[i]* (p+i));
```



```
int *p;
```

```
p = malloc(105 * sizeof(int));
```

```
for(i=0; i<105; i++)
```

```
scanf("/d", p+i);
```

```
for(i=0; i<105; i++)
```

```
printf("/d", → P[i]*(p+i));
```

```
int n, *p, i;
```

```
printf("Enter no. of elements");  
scanf("/d", &n);
```

```
p = malloc(n * sizeof(int));
```

```
if(p != NULL)
```

```
{
```

```
for(i=0; i<n; i++)
```

```
scanf("/d", p+i);
```

```
for(i=0; i<n; i++)
```

```
printf("/d", *(p+i));
```

```
}
```

